

A Mechanically Verified Translation of $MTL_{0,\infty}$ into Timed Büchi Automata

Elie Fares, Jean-Paul Bodeveix, Hussain Al-Aqrabi, and Azar Salami



Abstract—We mechanize a soundness and completeness theorem for a specified translation from pointwise dense-time $MTL_{0,\infty}$ to event-guarded timed Büchi automata, without location invariants and with existential initial clock valuations. The mechanized theorem covers well-formed core formulas: lower and strict upper bounds are positive, and non-strict upper bounds are nonnegative. We describe reductions for zero lower bounds and zero strict upper bounds, but do not mechanize this preprocessing. Ordinary timed operators are expressed through hatted Until and Release primitives, encoded in clocked LTL, and passed to an LTL-to-Büchi backend satisfying an explicit semantic contract. Relaxation and reset completion yield a timed Büchi automaton. The proof assigns one clock to each distinct primitive timed subformula, including overlapping activations. Soundness holds for every clock-consistent reset trace; completeness constructs a canonical trace using residual obligations. These arguments establish `MTL_to_TBA_correct_with`. The result covers strict and non-strict bounds within the stated core domain; it is not an integrated compiler or a performance comparison. For the alarm-response instance, we prove in Rocq that a propositional Büchi record manually transcribed from a Spot 2.16 graph has the same language as the translated formula over all propositional words, then instantiate the timed theorem for that record. This does not verify Spot or the graph-to-record transcription.

Index Terms—Metric temporal logic, timed Büchi automata, Rocq, formal verification, clocked LTL, semantic preservation.

1 INTRODUCTION

Temporal-property translators are part of software verification infrastructure: changing the timed language can make a downstream verifier accept a violating execution or reject a valid one. For bounded response, $\Box(req \Rightarrow \Diamond_{\leq d} ack)$, overlapping requests create pending deadlines while reusing one clock for that primitive timed formula. An incorrect reset at a later request could make an older deadline appear younger. Table 2 illustrates this general correctness risk; it does not allege a defect in prior tools.

Section 5.4 instantiates this assurance for alarms that must be handled within three time units. Rocq proves the backend contract for a manual transcription of a Spot 2.16 [1] graph, applies the timed theorem, and rejects a fixed violating trace; the graph-to-record link is outside the proof.

The mathematical translation result substantially pre-dates this paper. Bulychev et al. prove exact timed-automaton language results for safety and co-safety fragments; Li et al. state full $MTL_{0,\infty}$ language equality for their timed-Büchi construction (Theorem 12) and prove a best-choice reset policy language-preserving (Theorem 13) [2], [3]. Those constructions already use clocks indexed by timed subformulas, obligation states, and reset/unchanged markers. We claim no new translation algorithm, clock-allocation principle, or semantic property. The contribution is a Rocq-checked derivation of the established construction, factored into clocked-LTL correctness, relaxation, reset completion, and a backend contract. Its path-indexed induction and canonical-trace completeness make the proof obligations explicit, but this paper does not establish that these invariants form a reusable general proof method.

The formal domains also differ: the Rocq model uses real-valued bounds under the stated well-formedness restrictions, strictly increasing timestamps, and existential initial clock values; Li et al. use integer bounds, allow equal timestamps, and start clocks at zero. Thus the present theorem is not a direct refinement of CASAAL’s input/output semantics. The real-bound statement is a mathematical parameterization, not executable support for arbitrary real constants.

The factorization is independent of the LTL-to-Büchi translator if it satisfies the contract on all propositional words. Rocq 9.0.1 checks soundness and completeness for all eight strict and non-strict hatted primitives, plus relaxation and reset completion, on well-formed Rocq formulas with real-valued bounds. Zero-bound preprocessing, parsing, backend integration, output printing, and an executable compiler are outside the mechanization.

The broader research objective is to derive reusable Event-B refinement patterns from MTL properties, using TBA as an intermediate representation. This article establishes the verified MTL-to-TBA stage of that direction; transforming the resulting automata into Event-B patterns is outside its present results.

The contributions are verification results for this established construction:

- machine-checked soundness and completeness of the clocked-LTL factorization, with soundness for every clock-consistent reset trace and completeness through one canonical trace for all formula-keyed clocks and repeated activations;

Elie Fares, Hussain Al-Aqrabi, and Azar Salami are with the Higher Colleges of Technology, Sharjah, United Arab Emirates (e-mail: efares@hct.ac.ae; halaqrabi@hct.ac.ae; asalami@hct.ac.ae).

Jean-Paul Bodeveix is with IRIT, Université de Toulouse, France (e-mail: bodeveix@irit.fr).

- mechanized relaxation and reset completion, composed with an explicit language-equivalence contract for the propositional backend to obtain the event-guarded TBA theorem;
- a concrete alarm-response instance that proves this contract for a manual Rocq transcription of a Spot 2.16 graph and rejects a fixed violating timed trace.

The development includes the results `EncodingCorrect_proved`, `T_uses_every_clock`, and `MTL_to_TBA_correct_with`. The complete arguments for the canonical policy and its strict-bound cases appear in the supplementary material. A related unpublished companion manuscript [4] reports reuse of this Rocq translation proof and additional TBA optimization and UPPAAL-export results. No stable public version is available, so those downstream results are outside the evidence and scope of this paper.

For verification-tool developers, the theorem is a modular reference contract: a well-formed core Rocq formula and a propositional backend satisfying Assumption 4.1 yield an event-guarded TBA with the same timed language under existential initial clock values. A future implementation can use the Rocq development as a reference; transferring the guarantee requires separately establishing that its formula mapping, backend, and output semantics meet these interfaces. Parsing, bound encoding, backend invocation and validation, export, and adaptation to zero-initialized clocks are outside the theorem. The canonical trace is a completeness witness, not a runtime reset policy; no comparative clock-count, runtime, or automaton-size claim is made. The Spot script regenerates only the propositional graph, and the worked TBA is hand-derived; neither is an end-to-end checker run.

2 LOGIC AND SEMANTIC MODEL

2.1 Pointwise MTL_{0,∞}

Let Σ be a finite set of actions. Formulas are in negation normal form and use Boolean operators, next, untimed Until and Release, and timed Until and Release whose intervals have one endpoint at zero or infinity. We write a bound as $\sim d$, with $\sim \in \{<, \leq, >, \geq\}$. The logic also contains the hatted operators $U_{\sim d}$ and $\hat{R}_{\sim d}$. In a hatted operator the current position is skipped by the logical interval, while its timestamp remains the timing reference. We use $\hat{\Box}_{\sim d}\varphi$ as shorthand for $\perp \hat{R}_{\sim d}\varphi$; it requires φ at each strictly future position whose elapsed time from the current position satisfies the bound.

A timed word is $w = (a_i, t_i)_{i \in \mathbb{N}}$, where $a_i \in \Sigma$, $0 \leq t_0 < t_1 < \dots$, and timestamps diverge. We write $\text{elap}_w(i, j) = t_j - t_i$. For Until, $w, i \models_{\text{MTL}} p U_{\sim d} q$ holds when some $j \geq i$ satisfies q and $t_j - t_i \sim d$, with p at each position in $[i, j)$. For hatted Until, the witness is $j > i$ and p is required on (i, j) . Release is the dual universal condition: at every position in the bounded region, either q holds or p held earlier in the corresponding interval. Hatted Release excludes the current position from that region. We use the standard pointwise semantics [5].

For precision, the four bounded clauses used below are:

$$\begin{aligned}
 w, i \models_{\text{MTL}} p U_{\sim d} q &\iff \exists j \geq i. w, j \models_{\text{MTL}} q \\
 &\quad \wedge t_j - t_i \sim d \\
 &\quad \wedge \forall k \in [i, j). w, k \models_{\text{MTL}} p, \\
 w, i \models_{\text{MTL}} p \hat{U}_{\sim d} q &\iff \exists j > i. w, j \models_{\text{MTL}} q \\
 &\quad \wedge t_j - t_i \sim d \\
 &\quad \wedge \forall k \in (i, j). w, k \models_{\text{MTL}} p, \\
 w, i \models_{\text{MTL}} p R_{\sim d} q &\iff \forall j \geq i. \\
 &\quad \left(t_j - t_i \sim d \Rightarrow \right. \\
 &\quad \left. \left(w, j \models_{\text{MTL}} q \right. \right. \\
 &\quad \left. \left. \vee \exists k \in [i, j). w, k \models_{\text{MTL}} p \right) \right), \\
 w, i \models_{\text{MTL}} p \hat{R}_{\sim d} q &\iff \forall j > i. \\
 &\quad \left(t_j - t_i \sim d \Rightarrow \right. \\
 &\quad \left(w, j \models_{\text{MTL}} q \right. \\
 &\quad \left. \vee \exists k \in (i, j). w, k \models_{\text{MTL}} p \right) \right).
 \end{aligned}$$

The index intervals select event positions, whereas $t_j - t_i$ measures elapsed physical time. Thus, when a bound falls between two events, no event occurs exactly at that bound, and interval membership is still evaluated at the event positions. The strict increase assumption excludes simultaneous events; divergence is used in the lower-bounded Until fairness argument.

The Rocq development represents action names by natural numbers rather than a parameterized finite type, and timed words may use any natural-number name. Each formula mentions finitely many names, so its semantics and generated labels distinguish actions only by equality or inequality with those names. Equivalently, the construction operates over the finite quotient induced by those action tests; this is the finite alphabet used in the mathematical presentation.

The well-formedness condition requires $d > 0$ for every lower bound and strict upper bound, and $d \geq 0$ for a non-strict upper bound. Ordinary timed operators are definitions in the mechanized core. For upper bounds $\preceq \in \{\leq, <\}$ and lower bounds $\succeq \in \{\geq, >\}$, they are:

$$p U_{\preceq d} q \equiv q \vee (p \wedge (p \hat{U}_{\preceq d} q)), \quad (1)$$

$$p U_{\succeq d} q \equiv p \wedge (p \hat{U}_{\succeq d} q), \quad (2)$$

$$p R_{\preceq d} q \equiv q \wedge (p \vee (p \hat{R}_{\preceq d} q)), \quad (3)$$

$$p R_{\succeq d} q \equiv p \vee (p \hat{R}_{\succeq d} q). \quad (4)$$

These definitions separate the current-position case from the strictly future obligation. Their pointwise semantic compatibility, including the stated bound hypotheses, is proved in Rocq:

Theorem 2.1 (Semantic compatibility of the derived timed operators). *Under the well-formedness conditions above, Eqs. (1)–(4) agree with the standard pointwise semantics for all strict and non-strict upper and lower bounds.*

The identities follow by separating the current-position case from a strictly future endpoint. For upper Until, a witness at i contributes q ; every later witness requires p at i

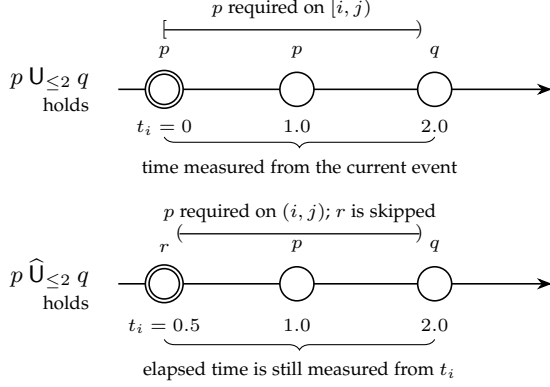


Fig. 1. Ordinary and hatted Until. The double circle is the timing reference. The hatted operator skips the current event logically but measures from its timestamp.

and leaves the hatted obligation. A lower bound with $d > 0$ excludes i , so p is required now and the remaining witness is hatted. The Release identities are the dual case split: an upper-bound region includes i and therefore requires q there, while a lower-bound region excludes i . Strict upper bounds also require $d > 0$, avoiding the degenerate < 0 case. The eight corresponding Rocq lemmas are identified in the result map in Supplement 3. Thus only the eight hatted timed constructors require primitive translation clauses. If a source formula has a zero lower bound, strict increase of timestamps makes every strictly future elapsed time positive. At the hatted level, $\hat{U}_{\geq 0}$ and $\hat{U}_{> 0}$ therefore reduce to $X(p \cup q)$, while $\hat{R}_{\geq 0}$ and $\hat{R}_{> 0}$ reduce to $X(p \cap q)$. For ordinary operators, the current-position case must also be simplified: the ≥ 0 cases are the corresponding untimed Until and Release operators, and the > 0 cases reduce to $p \wedge X(p \cup q)$ and $p \vee X(p \cap q)$, respectively. These reductions rely on the paper's strictly increasing event times and must be applied before encoding a source formula as a Rocq core formula; the preprocessing itself is not mechanized. A zero strict upper bound has an empty interval: Until reduces to $\perp$ and Release to $\top$; strict upper-bound constructors in the core consequently require $d > 0$. For the non-strict upper bound zero, the hatted future operators reduce to $\hat{U}_{\leq 0} \equiv \perp$ and $\hat{R}_{\leq 0} \equiv \top$, because all strictly future events have positive elapsed time. The derived ordinary operators therefore both reduce to their right operand: $p \cup_{\leq 0} q \equiv q$ and $p \cap_{\leq 0} q \equiv q$. These reductions make the boundary explicit: the mechanized domain excludes zero lower bounds and zero strict upper bounds, but includes non-strict upper bound zero.

This distinction is useful for sporadicity: if consecutive occurrences of e must be separated by at least θ , one may write $\Box(e \Rightarrow \hat{\Box}_{< \theta} \neg e)$. The hatted form retains the triggering event as the time origin; ordinary bounded temporal operators do not express this particular reference shift directly. Related pointwise MTL constructions and the CASAAL translation are discussed in prior work [3], [6]–[8].

2.2 Timed Büchi Automata

The verified target is an event-guarded TBA $\mathcal{A} = (L, \ell_0, X, E, Acc)$. Here L is a finite set of locations, ℓ_0 the

initial location, X a finite set of clocks, $E \subseteq L \times \text{Lab} \times \text{Guard} \times 2^X \times L$ is the set of action-labelled transitions with guards and reset sets, and $Acc \subseteq L$ the Büchi set. A run is a sequence $(\ell_i, v_i)_{i \geq 0}$ starting at ℓ_0 ; it follows one transition per event and visits Acc infinitely often. If the selected edge at event i is $(\ell_i, \alpha_i, g_i, Z_i, \ell_{i+1})$, then the event action must satisfy α_i and the valuation v_i must satisfy every guard in g_i . There are no location invariants in this model: timing constraints are checked at the events of the timed word. This is the TBA record represented in Rocq; the theorem does not cover a richer invariant-bearing automaton model without an additional encoding.

Let v_i be the clock valuation tested at event i and $\Delta_i = t_{i+1} - t_i$. A transition at event i tests v_i before its reset decision; the next valuation is

$$v_{i+1}(x) = \begin{cases} \Delta_i, & x \in Z_i, \\ v_i(x) + \Delta_i, & \text{otherwise.} \end{cases}$$

The initial nonnegative valuation v_0 is existentially quantified, as in the Rocq clock-consistent-extension semantics; standard zero initialization is not assumed.

3 Clocked-LTL Translation

3.1 Shared clock identities

The translation is structural. It maps Boolean and untimed temporal operators homomorphically and maps each primitive timed subformula to an LTL formula over an extended alphabet. For each action a , the extended alphabet contains distinct action atoms $\text{LAct}(a)$ and $\text{LNAct}(a)$, as well as clock-related atoms: clock comparisons $x \sim d$, a reset atom $\text{rst}(x)$, and a preservation atom $\text{unch}(x)$. Under timed-word interpretation, the action atoms mean that the current action is a or is different from a , respectively. The atom $\text{LNAct}(a)$ and the negated literal of $\text{LAct}(a)$ have the same timed-word interpretation, but remain distinct propositions to the unconstrained LTL backend. The reset and preservation atoms describe the clock update at the current event. Clock values are read before the event's reset, matching the guard semantics of a timed automaton.

For formula f , let $\text{Clock}(f)$ be its finite set of distinct primitive hatted timed subformulas. The clock associated with $F \in \text{Clock}(f)$ is $x = F$ itself. Clock identity uses structural equality on the Rocq MTL syntax tree, with bounds compared by equality in $\mathbb{R}$. Constructor, operands, comparison, and bound all participate: identical formula values at different paths share a clock, while semantic equivalence alone does not. No simplification-based merging is used. The main result proves that this clock also serves all dynamic activations of its subformula. The clock count is therefore bounded by the number of distinct primitive timed subformulas, which is at most the number of timed occurrences in f . If structural subformulas are represented as shared nodes, the translation can share their LTL encodings as well; this does not remove the usual worst-case growth of an LTL-to-Büchi construction.

This clock-sharing policy is distinct from automaton-state minimization. The reset trace is chosen for each formula identity as a whole. In particular, the proof cannot justify the clock bound by assigning a fresh clock to each

activation and subsequently identifying those clocks: it establishes directly that a single trace of reset decisions for the formula is sufficient.

3.2 Eight primitive clauses

Let F be a primitive hatted formula at path π , with operands p, q . Write $x = \text{clock_of}(F)$, $P = \mathcal{T}_{\pi 0}(p)$, $Q = \mathcal{T}_{\pi 1}(q)$, and define $\alpha W \beta = (\alpha \cup \beta) \vee G\alpha$. For upper comparisons $\lesssim \in \{\leq, <\}$ and lower comparisons $\gtrsim \in \{\geq, >\}$, complements are $\leq^c = >$, $<^c = \geq$, $\geq^c = <$, and $>^c = \leq$. The four clause shapes, each instantiated for its strict and non-strict bound, give all eight primitive cases:

$$\mathcal{T}_{\pi}(p \hat{U}_{\lesssim d} q) = X(((x \lesssim d) \wedge \text{unch}(x) \wedge P) \cup ((x \lesssim d) \wedge Q)), \quad (5)$$

$$\mathcal{T}_{\pi}(p \hat{U}_{\gtrsim d} q) = \text{rst}(x) \wedge X((P \cup ((x \lesssim d) \wedge (P \cup Q))) \vee (GP \wedge GFQ)), \quad (6)$$

$$\mathcal{T}_{\pi}(p \hat{R}_{\lesssim d} q) = \text{rst}(x) \wedge X(Q \cup W((x \lesssim^c d) \vee (P \wedge Q))), \quad (7)$$

$$\mathcal{T}_{\pi}(p \hat{R}_{\gtrsim d} q) = X(((x \gtrsim^c d) \wedge \text{unch}(x)) \cup W((P \cup Q) \vee ((x \gtrsim^c d) \wedge P))). \quad (8)$$

All clock atoms occur positively. Upper-bounded hatted Until uses strong Until, matching the mechanized clause: its witness must satisfy q while the clock remains within the upper bound. On a clock-consistent trace over divergent time, an infinite continuation cannot keep the preservation guard true forever. Lower-bounded Until retains its fairness branch, including GFQ , so that eventuality is not lost. The weak-Until clauses mark the end of bounded Release windows. Strict comparisons are represented directly; no strict bound is weakened by adjusting its constant.

The clauses have four distinct temporal roles. For upper-bounded Until, the clock is tested before any new reset: while its value remains in the allowed window, P must continue until Q appears. If the bound is passed first, no bounded witness exists and the strong-Until clause fails. For lower-bounded Until, a finite path waits for the threshold and then starts the unbounded $P \cup Q$ obligation. If the threshold is never reached, the second disjunct requires P at every position and Q infinitely often. Time divergence is what makes this infinite branch equivalent to a witness at distance at least d .

For upper-bounded Release, the initial Q is required and the weak-until stops either at the complementary clock comparison or when $P \wedge Q$ discharges the obligation. For lower-bounded Release, the clock remains unchanged before the threshold; after that point, the translated obligation is ordinary Release, with the endpoint alternative covering the transition into the threshold region. Thus Until clauses use existential witnesses, whereas Release clauses maintain a universal condition over the bounded region. The hatted shift is encoded by the outer X in every clause; for restart families, the separate $\text{rst}(x)$ atom denotes the reference timestamp at the current position.

TABLE 1
Clock comparisons used by each hatted primitive.

Primitive role	Non-strict bound	Strict bound
$\hat{U}_{\lesssim d}$ endpoint	$x \leq d$	$x < d$
$\hat{U}_{\gtrsim d}$ threshold	$x \geq d$	$x > d$
$\hat{R}_{\lesssim d}$ window end	$x > d$	$x \geq d$
$\hat{R}_{\gtrsim d}$ pre-threshold	$x < d$	$x \leq d$

TABLE 2
Finite prefix showing how a per-request reset can hide an expired older deadline. Ages are measured immediately before each event; the left column preserves the oldest request clock, while the right resets at each request. The deadline is five time units, so no continuation can repair the miss at time 5.

Event i	Action	Time t_i	Oldest age	Reset age
0	<i>req</i>	0	0	0
1	<i>req</i>	4	4	4
2	<i>ack</i>	6	6	2

3.3 Why hatted operators and shared clocks matter

The hatted operators preserve the triggering timestamp while moving logical evaluation to the next event. The same distinction appears in the clocked encoding: the initial X moves the temporal obligation forward, while the clock retains the reference selected by its reset trace.

For instance, $\Box(e \Rightarrow \hat{\Box}_{<\theta} \neg e)$ measures the exclusion window from the triggering event. Prefixing the ordinary operator with $\hat{\Box}$ shifts the reference to the next event, while omitting $\hat{\Box}$ also constrains the triggering event itself. The three readings differ on the same timed word, as shown in Fig. 2.

Clock sharing has two layers. First, identical primitive formulas at different syntax-tree paths share one clock by construction. Second, that single clock must represent all pending activations. For example, in $\Box(\text{req} \Rightarrow \hat{\Diamond}_{\leq 5} \text{ack})$, requests at times 0 and 4 overlap. If an acknowledgment occurs at time 6, resetting at the second request makes its age appear to be two, although the first request's deadline expired at time 5. The shared-clock construction retains the older reference and does not accept this word on the basis of the newer request alone; Table 2 records both ages. It is a finite prefix: every infinite continuation still has the missed deadline for the request at time 0. More generally, for any newer request at time t_k between an older activation t_i and a witness t_j , $t_j - t_k \leq t_j - t_i$; an endpoint that satisfies the older upper-bound obligation also satisfies the newer one. The argument depends on the required P condition holding between activations and the endpoint, which is exactly what the Until residual records.

For the other primitive families, the canonical policy and residual predicates handle the direction in which restarting or preserving the clock is required (Supplement 2.2).

3.4 Canonical reset policy

The encoding quantifies over clock traces. To prove completeness with one clock for a formula that may be activated arbitrarily often, it is enough to construct one trace

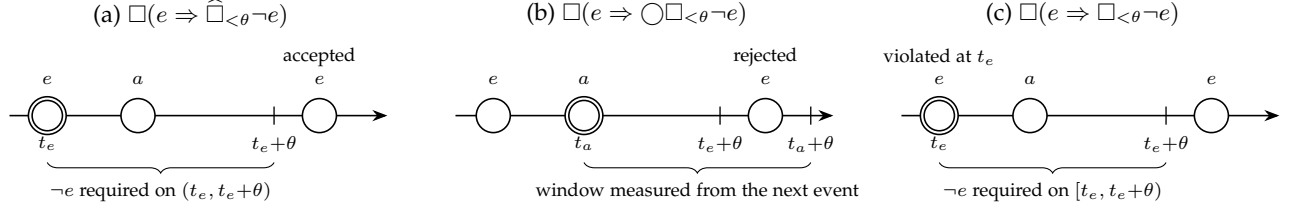


Fig. 2. The hatted sporadicity property and two ordinary-operator substitutions. A leading next operator shifts the timing reference to the following event; omitting it includes the triggering event in the window.

that gives priority to an older activation only while that activation remains useful. The proof records usefulness by residual obligations. For upper-bounded Until, the residual says that a witness is still reachable from the current clock age:

$$\begin{aligned} \text{URes}_{\lesssim}(i, v, d, p, q) \iff & \exists j \geq i. v + \text{elap}_w(i, j) \lesssim d \\ & \wedge w, j \models_{\text{MTL}} q \\ & \wedge \forall k \in [i, j]. \\ & \quad w, k \models_{\text{MTL}} p, \end{aligned}$$

where $\lesssim$ is either $\leq$ or $<$. For lower-bounded Release, the residual records that every position beyond the threshold is still covered:

$$\begin{aligned} \text{RRes}_{\geq}(i, v, d, p, q) \iff & \forall j \geq i. d \leq v + \text{elap}_w(i, j) \Rightarrow \\ & \left(w, j \models_{\text{MTL}} q \right. \\ & \quad \left. \vee \exists k \in [i, j]. \right. \\ & \quad \left. w, k \models_{\text{MTL}} p \right), \\ \text{RRes}_{>}(i, v, d, p, q) \iff & \forall j \geq i. d < v + \text{elap}_w(i, j) \Rightarrow \\ & \left(w, j \models_{\text{MTL}} q \right. \\ & \quad \left. \vee \exists k \in [i, j]. \right. \\ & \quad \left. w, k \models_{\text{MTL}} p \right). \end{aligned}$$

The strict predicates preserve the strict comparison itself. They are defined and shifted in the Rocq development (the full definitions and one-step lemmas are in Supplement 2.2).

The residual predicates are deliberately asymmetric. An upper-Until residual is existential: one still-reachable endpoint is enough, and that endpoint can discharge the obligation initiated at the current position. A lower-Release residual is universal: every position that has entered the threshold region must satisfy the Release condition. The canonical policy preserves a clock only when its residual remains true, so clock reuse follows from the semantic shape of the obligation rather than a syntactic rule such as “reset on every activation.” This distinction is needed for repeated activations, but the policy is not part of the generated automaton: it is a word-dependent witness used only to prove completeness.

For clock $x = F$ with current value v , the canonical policy is summarized in Table 3. “Preserve iff” means that the clock is not reset exactly under the listed condition; in the remaining cases it is reset. For the restart families, the stated test is the reset condition. The canonical extension

TABLE 3
Canonical reset policy for each timed-operator family.

Primitive clock F	Reset decision at position i
$p \hat{U}_{\leq d} q$	Preserve iff $v \leq d$, $\text{URes}_{\leq}(i, v, d, p, q)$, and $w, i \not\models_{\text{MTL}} q$.
$p \hat{U}_{< d} q$	Preserve iff $v < d$, $\text{URes}_{<}(i, v, d, p, q)$, and $w, i \not\models_{\text{MTL}} q$.
$p \hat{U}_{\geq d} q, p \hat{U}_{> d} q, p \hat{R}_{\leq d} q,$ $p \hat{R}_{< d} q$	Reset iff $w, i \models_{\text{MTL}} F$.
$p \hat{R}_{\geq d} q$	Preserve iff $v < d$, $\text{RRes}_{\geq}(i, v, d, p, q)$, and $w, i \not\models_{\text{MTL}} p$.
$p \hat{R}_{> d} q$	Preserve iff $v \leq d$, $\text{RRes}_{>}(i, v, d, p, q)$, and $w, i \not\models_{\text{MTL}} p$.

uses this decision for the formula-keyed clock and updates its value by

$$u_0 = 0, \quad u_{i+1} = \begin{cases} \Delta_i, & \text{if the policy resets at } i, \\ u_i + \Delta_i, & \text{otherwise.} \end{cases}$$

The policy depends on truth in the timed word and, through residuals, on its future. It is a semantic witness used in the completeness proof, not an algorithm executed by the translator. Its input is the timed formula itself, not a syntax-tree path, so identical occurrences share the same reset trace.

The two preservation cases retain an older reference only if the residual is still true and the current event has not discharged it. A later endpoint that satisfies the older upper-bounded Until also satisfies newer pending copies with shorter remaining prefixes. For lower-bounded Release, an old reference reaches the threshold no later than a newer one; the residual shift $(i, v) \mapsto (i + 1, v + \Delta_i)$ carries the remaining obligation. Conversely, lower-bounded Until and upper-bounded Release restart when the current formula is true; their translated continuation keeps any older obligations represented. The strict cases follow the same organization with strict residuals and guards.

3.5 Why one trace suffices for shared activations

The clock key is the primitive timed formula, not its syntax-tree path or the dynamic activation that created an obligation. Thus identical subformulas at different tree positions use the same value and reset trace; the Rocq lemma `identical_nodes_share_canonical_trace` proves this equality. Dynamic activations still impose distinct semantic obligations. The canonical policy represents them by retaining an older clock reference only while its residual obligation remains valid.

For an upper-bounded Until, let activations at $i_0 \leq i_1 < j$ share an endpoint j . The older reference gives the stronger deadline constraint,

$$t_j - t_{i_1} \leq t_j - t_{i_0} \leq d.$$

Its required p -prefix contains the suffix required by the newer activation. Consequently, preserving the older clock while $\text{URes}_{\leq}$ holds covers both activations; when the older residual fails or its endpoint has discharged it, the policy can reset for a new obligation. This is the invariant used by `complete_UhatLe` and its strict counterpart.

Lower-bounded Release has the dual universal shape. An older reference has at least the age of a newer one and therefore reaches the lower threshold no later. The predicate RRes records whether every position beyond that threshold remains covered by q or an earlier p . While the clock is preserved, the residual advances by

$$(i, v) \mapsto (i + 1, v + \Delta_i),$$

which is the one-step lemma `rge_residual_shift` used by `complete_RhatGe`. In the dual restart families, a current activation resets the clock, while the Γ branch for lower-bounded Until and weak-until continuation for upper-bounded Release retain the older obligations in the LTL formula. The same cases use strict residuals and strict guards for the strict primitives. The resulting sharing argument is about semantic obligations, not an assumption that activations are nonoverlapping.

4 FROM CLOCKED LTL TO A TBA

4.1 LTL-to-Büchi contract

The backend treats extended atoms as independent propositions. It returns a propositional Büchi automaton whose language is exactly that of the clocked-LTL formula. The theorem is stated for any such backend, deterministic or not.

Assumption 4.1 (Backend correctness). *For each formula F and each propositional word s over its atoms, $\text{PBA_accepts}(A, s)$ iff $s, 0 \models F$, where A is the backend automaton for F .*

This contract ranges over all propositional words, including assignments that do not correspond to any clock-consistent timed word. That stronger scope is used by relaxation.

4.2 Relaxation and reset completion

The clocked-LTL formula has positive occurrences of clock-related atoms, so its propositional language is upward closed in those atoms. In each backend cube, relaxation deletes negative literals over clock comparisons and reset/preservation markers and removes irrelevant clock-related atoms; contradictory cubes are discarded. Action literals are retained as transition labels. For soundness, first reconstruct an assignment satisfying the selected original cube and apply the backend contract to that word. Upward closure then transfers satisfaction to the relaxed word, which can only add truth on relevant clock-related atoms. The position-by-position reconstruction is given below.

Each clock belongs to one of two classes. A preservation clock is used with upper-bound tests and $\text{unch}(x)$, and a

restart clock is used with lower-bound tests and $\text{rst}(x)$. Reset completion converts a relaxed cube into a TBA transition as follows:

- reset a preservation clock unless the cube contains $\text{unch}(x)$;
- reset a restart clock exactly when the cube contains $\text{rst}(x)$;
- remove reset and preservation markers, retaining clock tests as the transition guard and action literals as its label.

The asymmetry is essential. Additional resets only decrease preservation clock values and hence preserve upper tests. A restart clock can be reset only when explicitly required, since decreasing it could invalidate a lower test. The full monotonicity argument is given in Supplement 1.1.

Lemma 4.2 (Reset completion). *For every accepting run of the relaxed backend automaton on a clock-consistent extension, there is a clock-consistent extension with the same timed word and initial values whose resets are exactly those prescribed by reset completion, and the same run is accepting in the resulting TBA.*

Proposition 4.3 (Exactness of relaxation and reset completion). *Let f be well formed and let A satisfy Assumption 4.1 for $\mathcal{T}(f)$. The TBA obtained from A by relaxation and reset completion accepts exactly the timed words w for which $w, 0 \models_{\text{MTL}} f$, under existential initial clock values.*

The relaxation proof uses positivity of the clock atoms in $\mathcal{T}(f)$. A run of the original Büchi automaton remains a run after relaxation. Conversely, take a run of the relaxed automaton and, at each position, reconstruct an assignment satisfying the selected original backend cube: make each extended atom false when its deleted negative literal requires this, and choose values for removed irrelevant atoms as needed by that cube. The backend contract then makes the reconstructed word a model of $\mathcal{T}(f)$. It agrees with the relaxed word on base atoms, while the relaxed word can only add truth on relevant extended atoms. Upward closure of $\mathcal{T}(f)$ therefore makes the relaxed word a model as well. Reset completion then assigns resets monotonically: preservation clocks become no larger, keeping upper guards true, and restart clocks become no smaller, keeping lower guards true. Thus the same Büchi run remains accepting. The proof is mechanized as `relax_sound`, `relax_complete`, and `complete_complete`.

Acceptance convention. The theorem quantifies existentially over the nonnegative clock values at the first event. This is a property of the accepted timed language; it does not by itself establish equivalence with an automaton whose clocks must start at zero. For example, a first transition guarded by $x \geq 1$ may be enabled for one existential initial value and disabled when the first event is at time zero. The worked example avoids dependence on that value by resetting its deadline clock on the first transition. The conventional initialization question is otherwise outside Theorem 5.3.

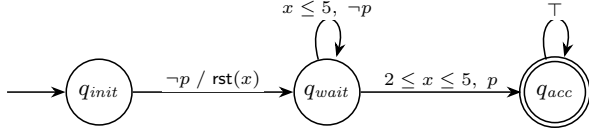


Fig. 3. Hand-simplified equivalent event-guarded TBA for f . The initial transition requires $\neg p$ and resets x ; in the waiting location, $x \leq 5$, and acceptance also requires $x \geq 2$. Thus p first holds at age in $[2, 5]$.

4.3 Worked example: bounded eventuality with an exclusion window

Consider

$$f = (F_{\leq 5} p) \wedge (G_{< 2} \neg p).$$

It requires p to be false at the origin and until age two, then true by age five. Figure 3 shows a hand-simplified equivalent one-clock TBA, not the syntax-directed output, which assigns separate clocks to the two distinct timed subformulas. The steps below show the clocked-LTL translation and timed reinterpretation.

Step 1: The formula is in negation normal form. Expanding the derived operators and applying the timed-operator identities gives

$$\begin{aligned} f &= (\top \cup_{\leq 5} p) \wedge (\perp R_{< 2} \neg p) \\ &= (p \vee (\top \wedge \top \hat{\cup}_{\leq 5} p)) \\ &\quad \wedge (\neg p \wedge (\perp \vee \perp \hat{R}_{< 2} \neg p)). \end{aligned}$$

These current-position cases follow Eqs. (1) and (3).

Step 2: The two distinct primitive timed subformulas receive clocks x and y , respectively. Their clocked-LTL translation is

$$\begin{aligned} \Phi_U &= p \vee (\top \wedge X(((x \leq 5) \wedge \text{unch}(x) \wedge \top) \cup \\ &\quad ((x \leq 5) \wedge p))), \\ \Phi_R &= \neg p \wedge (\perp \vee (\text{rst}(y) \wedge \\ &\quad X(\neg p \vee ((y \geq 2) \vee (\perp \wedge \neg p))))) , \\ \mathcal{T}(f) &= \Phi_U \wedge \Phi_R. \end{aligned}$$

The Until preserves x ; the Release test $y \geq 2$ complements the source bound < 2 , allowing p from age two.

Step 3: A Spot backend output for $\mathcal{T}(f)$ is shown on the left of Fig. 4. Clock guards, reset and preservation markers, and action atoms are independent propositions to the backend. In particular, $p = \text{LAct}(p)$ and $\neg p = \text{LNAct}(p)$ are distinct atoms, while $!p$ is the negated literal of $\text{LAct}(p)$; a backend cube may contain both p and $\neg p$. Timed reinterpretation imposes action consistency.

Step 4: Relaxation removes negative literals over extended clock atoms. Reset completion resets x unless $\text{unch}(x)$ is present and resets y exactly when $\text{rst}(y)$ is present. The right side of Fig. 4 shows the transitions after marker removal; later optimization removes unsatisfiable action cubes and simplifies the automaton.

5 CORRECTNESS AND CLOCK BOUND

An extended word ρ adds, at every position i , a value $\text{val}_\rho(i, x) \in \mathbb{R}_{\geq 0}$ and reset decision $\text{reset}_\rho(i, x)$ for each $x \in \text{Clock}(f)$. It is clock-consistent when, for $\Delta_i = t_{i+1} - t_i$,

$$\text{val}_\rho(i+1, x) = \begin{cases} \Delta_i, & \text{reset}_\rho(i, x), \\ \text{val}_\rho(i, x) + \Delta_i, & \text{otherwise.} \end{cases} \quad (9)$$

The value at position i is the value tested by guards before the reset decision at that position. Let $\text{same_base}(\rho, w)$ mean that w is the timed-word projection of ρ .

Theorem 5.1 (Correctness of the clock encoding). *For every well-formed formula f and timed word w ,*

$$\begin{aligned} w, 0 \models_{\text{MTL}} f &\iff \exists \rho. \text{same_base}(\rho, w) \\ &\quad \wedge \text{clock_consistent}(\rho) \\ &\quad \wedge \rho, 0 \models_{\text{LTL}} \mathcal{T}(f). \end{aligned}$$

The existential quantifier over ρ is taken jointly over all distinct primitive timed subformulas. Soundness does not rely on choosing the canonical trace: each clock may follow any reset pattern allowed by the encoding, provided its value recurrence is respected. Completeness is the more delicate half because all activations that share a formula-keyed clock must be served simultaneously. The proof constructs the clock traces in parallel, using the residual predicates only for the preservation families and the direct restart rule for the remaining families. This result is the Rocq theorem `EncodingCorrect_proved`. Its soundness direction holds for any clock-consistent reset trace. Completeness constructs one canonical trace for all clocks together. The detailed induction, including the strict-bound arithmetic, is in Supplement 2.

Corollary 5.2 (One clock per distinct primitive timed subformula). *All occurrences of the same primitive timed subformula use the same logical clock. The encoding uses exactly the clocks in $\text{Clock}(f)$, hence at most one clock per distinct primitive timed subformula, independently of the number of syntax-tree occurrences or run-time activations.*

Proof. Every clock in $\mathcal{T}(f)$ is indexed by an element of $\text{Clock}(f)$, and `T_uses_every_clock` proves that every such element occurs in $\mathcal{T}(f)$. A single extension assigns one value and reset trace to each element of $\text{Clock}(f)$. Theorem 5.1 proves that the canonical trace suffices for every pending activation of that formula. $\square$

Define `compile_with(A)` by applying relaxation and reset completion to backend automaton A . Timed acceptance is existential over clock-consistent extensions of the timed word and requires that reset decisions agree with the reset sets along an accepting run.

Theorem 5.3 (MTL-to-TBA correctness). *For every well-formed f , backend automaton A satisfying Assumption 4.1, and timed word w ,*

$$w, 0 \models_{\text{MTL}} f \iff \text{TBA_accepts}(\text{compile_with}(A), w).$$

In Rocq this is `MTL_to_TBA_correct_with`; the backend correctness is an explicit hypothesis, not a project-specific axiom.

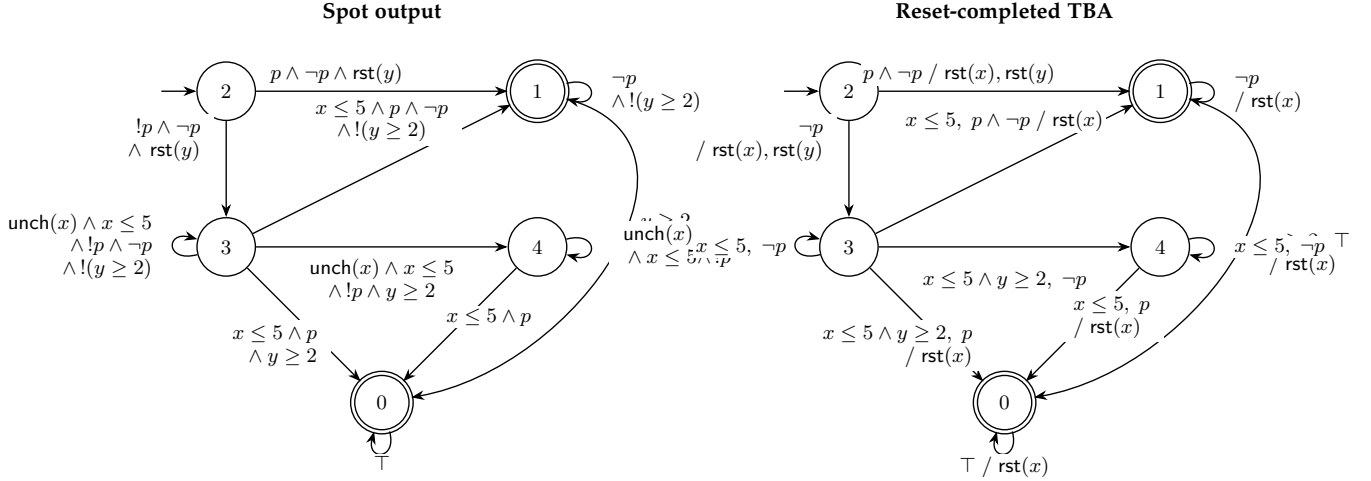


Fig. 4. Spot 2.16 output (redrawn, left) and hand-derived reset-completed TBA (right; same state IDs). Raw input and DOT are in `examples/spot/`. Right-hand edges show guards, actions, and resets; `rst(c)` and `unch(c)` mark reset and preservation.

Proof. Combine Theorem 5.1 with Proposition 4.3. The forward direction uses the canonical extension and reset-completion lemma; the reverse direction uses upward closure under relaxation, the backend contract, and clock-encoding soundness. $\square$

5.1 Semantic and arithmetic proof facts

The proof uses the standard LTL expansions of always, eventually, and weak Until. For every LTL formula A and position i ,

$$\begin{aligned} \rho, i \models_{\text{LTL}} GA &\iff \forall j \geq i, \rho, j \models_{\text{LTL}} A, \\ \rho, i \models_{\text{LTL}} FA &\iff \exists j \geq i, \rho, j \models_{\text{LTL}} A, \\ \rho, i \models_{\text{LTL}} GFA &\iff \forall n \geq i \exists j \geq n, \rho, j \models_{\text{LTL}} A, \end{aligned} \quad (10)$$

$$\begin{aligned} \rho, i \models_{\text{LTL}} A WB &\iff \left(\exists j \geq i, \rho, j \models_{\text{LTL}} B \right. \\ &\quad \wedge \forall k \in [i, j), \rho, k \models_{\text{LTL}} A \Big) \\ &\quad \vee \forall j \geq i, \rho, j \models_{\text{LTL}} A. \end{aligned} \quad (11)$$

These are the Rocq lemmas `LG_semantics`, `LF_semantics`, `LGF_semantics`, and `LW_semantics`. Time divergence also gives a future B position at any required physical distance when GFB holds (`GF_far`), which supplies the witness in the fairness branch of lower-bounded Until.

Clock consistency converts guards into elapsed-time bounds.

Lemma 5.4 (Accumulation under preservation). *If x is not reset in $[i, j)$, then*

$$\text{val}_\rho(j, x) = \text{val}_\rho(i, x) + \text{elap}_w(i, j).$$

Since clock values are nonnegative, $\text{elap}_w(i, j) \leq \text{val}_\rho(j, x)$. Thus an upper guard $x \leq d$ at j entails

$$\text{elap}_w(i, j) \leq d. \quad (12)$$

An upper guard $x < d$ gives the strict version.

Lemma 5.5 (Clock value after a reset). *If x is reset at i and $j \geq i + 1$, then $\text{val}_\rho(j, x) \leq \text{elap}_w(i, j)$. If it is not reset again before j , equality holds.*

A later reset only makes the clock younger. Hence a lower guard gives

$$x \geq d \implies d \leq \text{val}_\rho(j, x) \leq \text{elap}_w(i, j), \quad (13)$$

and $x > d$ entails a strict lower bound. The Rocq lemmas are `preserve_accumulate`, `after_reset_value_le_elapsed`, and `reset_preserve_value_eq_elapsed`.

5.2 Strict-bound cases

The eight hatted constructors are the primitive proof families. Ordinary timed operators have already been reduced by Eqs. (1)–(4). Strict comparisons remain strict through each translation and proof case. In particular, preservation gives $\text{elap}_w(i, j) \leq \text{val}_\rho(j, x)$, so $x < d$ implies $\text{elap}_w(i, j) < d$. After a reset, $\text{val}_\rho(j, x) \leq \text{elap}_w(i, j)$, so $x > d$ implies $\text{elap}_w(i, j) > d$.

The fairness branch of $\hat{U}_{>d}$ needs one additional step. Time divergence gives a future q at distance at least d from position $i + 1$; strict timestamp increase gives $t_{i+1} > t_i$. The same witness is therefore at distance strictly greater than d from i . This is the Rocq lemma `sound_UhatGt`.

5.3 Mechanization Challenges and Proof Architecture

The proof separates soundness from completeness. For soundness, every clock-consistent reset trace satisfying the clocked-LTL encoding implies MTL satisfaction. For completeness, each timed word satisfying the MTL formula admits an existential canonical clock-consistent trace satisfying the encoding. Relaxation and reset completion then preserve the timed language. The following points identify the Rocq invariants used for these obligations.

A. Occurrence-indexed induction with formula-keyed clocks. The recursive translation is written as $\mathcal{T}_\pi(f)$: its recursive calls extend the syntax-tree path π . Clock identity, however, depends on the exact primitive-formula syntax and is independent of that path: syntactically different primitive formulas have different keys, while identical formulas at distinct positions in the tree use the same clock value and reset trace. A root-only induction statement does not expose

the child-occurrence facts needed at each recursive call while retaining that root-wide clock identity. The development proves `canonical_encoding_all`, whose induction statement ranges over every path π for which $\text{node}(\text{root}, \pi) = f$. The lookup lemmas `node_at_left_binary` and `node_at_right_binary` establish the child paths; `identical_nodes_share_clock` and `identical_nodes_share_canonical_trace` show that equal formula keys denote one clock and one trace. This remains structural induction on f , with a stronger path-indexed invariant. The lemmas `T_at_uses_clocks` and `T_uses_every_clock` also show that the encoding uses exactly the formula-indexed clock set.

B. Completeness with repeated activations. One clock must support every activation of a primitive formula, including activations that overlap and have different remaining deadlines. Completeness cannot choose a separate clock trace for each activation; it must construct a single clock-consistent extension that satisfies all of them. For upper-bounded Until, `ule_residual` (and its strict counterpart `ult_residual`) records a still-available witness, its remaining deadline, and the required p -prefix. The preservation rule retains the older reference only while this residual remains true and q has not already discharged it. That reference imposes the stronger deadline and prefix constraint, so it also covers newer pending activations. For lower-bounded Release, `rge_residual` and `rgt_residual` record the universal obligation beyond the threshold: each such position must satisfy q or have an earlier p that discharges the obligation. The shift lemmas `rge_residual_shift` and `rgt_residual_shift` carry this condition across an event while the clock is preserved. These are the two preservation families. Lower-bounded Until and upper-bounded Release instead restart their clock at a satisfying activation; their translated temporal continuations retain older obligations. The eight completeness lemmas in Table 4 prove these cases, including their strict variants.

C. Relaxation and reset completion. The propositional backend treats clock tests and reset markers as independent atoms, so its language contract applies even to words that are not valuations of any clock-consistent timed trace. Relaxation deletes negative literals over these extended atoms. To recover a model of the clocked-LTL formula, the proof rebuilds at each position an assignment satisfying the original backend cube, then uses positivity of the translated formula to show that adding truth to extended atoms preserves satisfaction. The Rocq lemmas `psat_mono`, `relax_sound`, and `relax_complete` formalize this argument. Reset completion is a separate obligation because it fixes the actual reset set of each TBA edge. It may reset a preservation clock more often, producing no larger values and preserving upper-bound tests; for a restart clock it resets only when the cube requires it, producing no smaller values and preserving lower-bound tests. The local and run-level arguments are captured by `complete_trans_sound`, `complete_sound`, and `complete_complete`. The proof does not assume that the backend understands clocks.

D. Strict temporal boundaries. The strict constructors require proof obligations with $<$ and $>$ preserved through both the residual predicates and the clock-value inequalities. The strict residuals `ult_residual` and `rgt_residual`, to-

TABLE 4
Rocq lemmas for the primitive timed proof cases. Each strict and non-strict comparison has its own row.

Primitive family	Soundness	Canonical completeness
$\hat{U}_{\leq d}$	<code>sound_UhatLe</code>	<code>complete_UhatLe</code>
$\hat{U}_{\geq d}$	<code>sound_UhatGe</code>	<code>complete_UhatGe</code>
$\hat{R}_{\leq d}$	<code>sound_RhatLe</code>	<code>complete_RhatLe</code>
$\hat{R}_{\geq d}$	<code>sound_RhatGe</code>	<code>complete_RhatGe</code>
$\hat{U}_{< d}$	<code>sound_UhatLt</code>	<code>complete_UhatLt</code>
$\hat{U}_{> d}$	<code>sound_UhatGt</code>	<code>complete_UhatGt</code>
$\hat{R}_{< d}$	<code>sound_RhatLt</code>	<code>complete_RhatLt</code>
$\hat{R}_{> d}$	<code>sound_RhatGt</code>	<code>complete_RhatGt</code>

gether with `preserve_strict_upper_guard_sound` and `reset_strict_lower_guard_sound`, keep those boundaries explicit. One additional argument occurs in the fairness branch of $\hat{U}_{> d}$: time divergence and `GF_far` yield a q at distance at least d from position $i + 1$, and strict timestamp increase gives $t_{i+1} > t_i$, making the same witness strictly more than d after i (`sound_UhatGt`). The proof works over real-valued bounds in the stated well-formed domain; zero-bound preprocessing remains outside the mechanization. Table 4 lists the separate proof pair for each of the eight cases. These invariants document the work needed to mechanize this translation; formula/residual and reset-marker constructions have close mathematical precedents. The current development does not demonstrate proof reuse for another translator or extract an executable translation tool.

Artifact Availability. The public repository at <https://github.com/efares1/coq-verified-mtl-to-tba> hosts the Rocq development, alarm-response instance, Spot examples, and reproduction instructions. This revision adds executable checks for the saved DOT/record correspondence and a reset-risk scenario; these are included in the local submission candidate but have not been published to GitHub. The checks are not verified components. Rocq proves the record’s language contract, while Spot and the DOT-to-record link remain outside the proof. The GitHub URL is mutable; an immutable identifier for the exact snapshot should be inserted after the release is approved and publicly archived. This is not an executable compiler.

`EncodingCorrect_proved` collects the local cases; `MTL_to_TBA_correct_with` composes them under the explicit backend contract. The supplement gives the detailed proofs and theorem composition.

What the mechanization assumes. The proof boundary is summarized in Table 5. In particular, the correctness of the untimed backend is an explicit proposition-level assumption; the timed proof does not depend on Spot or on determinism. The theorem `MTL_to_TBA_correct_with` accepts backend language equivalence as an explicit argument. The convenience theorem `MTL_to_TBA_correct` instead uses the project axiom `LTL_TO_BUCHI_CORRECT`. The audit in `proof/ASSUMPTIONS.txt` records the standard Rocq library assumptions for real-number reasoning, functional extensionality, description, and classical logic. These are separate from the backend contract and the convenience theorem’s project axiom. More precisely, the audit

TABLE 5
Scope of the mechanized translation theorem.

Pipeline stage	Assurance boundary and software consequence
Formula front end	Input is a well-formed Rocq formula; parsing and finite bound encoding are outside the theorem.
Timed encoding	Rocq proves clocked-LTL equivalence using one clock per distinct primitive timed formula.
Propositional backend	The generic theorem assumes the backend contract; Rocq proves it for a manual transcription of one Spot 2.16 graph.
Graph correspondence	Python checks the saved DOT/record states, acceptance, and guards; importer correctness and Spot remain unverified.
TBA construction	Rocq proves relaxation and reset completion; output has event guards and no location invariants.
Model-checker adapter	Initial clock values are existential; zero initialization and export adapters are outside the theorem.
Integration	No end-to-end translator or timed-checker run; scripts demonstrate one graph correspondence and an illustrative reset fault.

for `EncodingCorrect_proved` lists only standard/classical assumptions; `MTL_to_TBA_correct_with` takes backend equivalence as an explicit premise; `MTL_to_TBA_correct` additionally depends on `LTL_TO_BUCHI_CORRECT`; and the alarm theorem has standard assumptions because its record contract is proved separately. Parsing, backend invocation, and output printing are outside this theorem.

5.4 Alarm-response proof instance

Consider $f_{alarm} = G(alarm \Rightarrow \Diamond_{\leq 3} handled)$, meaning every alarm must be handled within three time units. In `proof/OverlappingResponse_Example.v`, the formula is `MR MFalse (MOr (MNotAtom ALARM) (MULE 3 MTrue (MAtom HANDLED)))`, where `ALARM` and `HANDLED` are actions 1 and 2. Rocq checks that it is well formed and has one primitive timed subformula, $\hat{U}_{\leq 3} \top handled$, whose obligations share clock x . Abbreviate $LNAct(alarm)$ by N , $LAct(handled)$ by H , $x \leq 3$ by C , and $LUnch(x)$ by U . Spot receives the exact clocked-LTL formula

$$G(N \vee H \vee X((C \wedge U) \cup (C \wedge H))).$$

Spot 2.16's input and output are saved in `examples/spot/alarm_response_backend.ltl` and `examples/spot/alarm_response_backend.dot`. The generated graph has initial state 0, accepting set $\{0, 1\}$, and these transitions:

source	target	label
0	0	$H \vee N$
0	1	$\neg H \wedge \neg N$
1	0	$C \wedge H$
1	2	$C \wedge \neg H \wedge U$
2	0	$C \wedge H$
2	2	$C \wedge \neg H \wedge U$

The graph is manually transcribed as `alarm_response_spot_backend`; Rocq splits the disjunctive self-loop into two cubes. Lemma `alarm_response_T_eq` matches its input to

the Rocq translation; `alarm_response_spot_backend_contract` proves that the transcribed automaton has exactly that LTL language on all propositional words. This discharges the backend premise for this instance. The regeneration script requires `ltl2tgba --version` to report Spot 2.16.

The resulting concrete TBA is `alarm_response_spot_tba`. Theorem `alarm_response_spot_tba_correct` instantiates Theorem 5.3 and proves that it accepts exactly the timed words satisfying f_{alarm} . Its compiled transition listing is included in the supplementary material for inspection.

For the fixed infinite trace with alarms at times 0 and 2 and a handled event at time 4, `alarm_response_trace_violates` proves that the first alarm misses its deadline. Corollary `alarm_response_trace_rejected_by_spot_tba` proves that this TBA rejects it. The graph regenerates with the Spot 2.16 command in `examples/spot/regenerate.sh`. No graph importer, timed model checker, or TBA export is verified or run. The repository's Python consistency check matches the saved graph and Rocq record up to state renaming, including all guards; mutation self-tests reject a changed guard and accepting set. A separate illustrative scenario shows how an intentionally incorrect reset at each request can accept this trace. These executable checks do not establish universal correctness or make a claim about CASAAL.

6 RELATED WORK

Pointwise MTL has extensive decidability and expressiveness results [5]–[7], [9], [10]. Unrestricted dense-time MTL has no general effective translation to finite ordinary timed automata; bounded-variability and alternating-automaton results have different scopes [11], [12]. $MTL_{0,\infty}$ retains common response and separation properties [2], [10]. Our hatted operators make the timing reference explicit in the logic and proof.

Translations of timed logics. As Table 6 shows, the full-fragment language-equivalence theorem already appears in Li et al. The overlap trace illustrates a generic reset risk, not a defect in prior tools; we do not claim their constructions or proofs omit overlapping obligations. Our result is a machine-checked derivation for a different formal model and proof boundary, not a new language-correctness property or reset algorithm.

MITL translations use temporal testers, automata networks, or one-clock alternating automata [12]–[14]; `MightyL` and `MITL2Timed` provide UPPAAL-oriented tools for that distinct logic [15], [16]. Obligation-merging techniques are related, while our one-sided residual predicates support the checked completeness proof.

Clocked temporal logic and subformula clocks. `CLTLoc` adds clock comparisons and reset predicates to LTL and has been used to encode metric logics [17]. Our similar propositional vocabulary is an automaton-synthesis intermediate: the backend treats clock tests and reset markers as propositions. Positivity gives upward closure, and reset completion restores timed meaning; hence the explicit relaxation proof.

Event-clock logics associate clocks with subformulas to measure time since or until events [11], [18], [19]. Formula-indexed clocks are established; our proof checks that one clock per distinct timed formula preserves language across

TABLE 6

Theorem and evidence comparison with the closest $MTL_{0,\infty}$ automata constructions. Bulychev et al. prove exactness for safety/co-safety; Li et al. state full-fragment language equality. No row ranks efficiency.

Work	Semantic scope	Existing correctness and construction	Mechanization and implementation evidence
Bulychev [2]	Pointwise non-Zeno infinite words; integer bounds, all four comparisons, nondecreasing timestamps.	Theorems 1–2 prove exact automaton languages for safety/co-safety. The construction already uses timed-subformula clocks, obligations, and reset/unchanged markers.	Mathematical proofs and controller-synthesis tooling; no mechanized full-fragment TBA theorem.
Li et al. [3]	Full $MTL_{0,\infty}$; pointwise non-Zeno words, integer bounds, all four comparisons, nondecreasing timestamps, zero initial clocks.	Theorem 12 states exact language equality for the timed-Büchi construction; Theorem 13 proves best-choice resets language-preserving.	CASAAL implements the workflow and deterministic approximations [8]; the proof is mathematical.
This paper	Well-formed Rocq formulas with real bounds, strictly increasing timestamps, existential nonnegative initial clocks.	<code>EncodingCorrect_proved</code> proves clocked-LTL equivalence for all eight primitive cases; TBA composition is conditional on backend equivalence.	Rocq-checked proof and one proved contract for a manual Spot transcription; no executable MTL compiler or performance comparison.

repeated activations and all strict and non-strict cases (Section 5.3).

Recent automata-based real-time verification. Recent neighboring work addresses different settings: an improved one-clock alternating MTL automaton for finite timed words with clock deactivation and zone analysis [20]; and TEMPORA’s pointwise MITL model checking with past and future modalities [21]. These do not translate our $MTL_{0,\infty}$ fragment to infinite-word TBAs. Ho et al.’s MightyPPL translates pointwise MITL with past and Pnueli modalities over finite and infinite timed words; it gives mathematical construction results and proof sketches alongside a complete implementation whose testers sequentialize overlapping obligations [22]. The QEST+FORMATS follow-up implements existentially punctual MTLPL [23], [24]. Its term *one-sided MTL* denotes the subfragment without past or Pnueli, whereas here it describes interval shapes $[0, d]$ and $[d, \infty)$. Any syntactic containment comparison is limited to conventional source formulas after zero-bound reduction and with integer bounds; it excludes hatted AST constructors. The QEST fragment also permits punctual and non-one-sided intervals in its stated positions. Its punctual-response example does not imply support for unrestricted punctual MTL; differing bounds and timed-word conventions preclude a shared-input semantic comparison.

These tools address different fragments or output models; this paper neither re-verifies them nor makes a shared-input, coverage, clock-count, or performance comparison. The Rocq development is not integrated with CASAAL, MightyPPL, or another checker.

Mechanized translations and trusted backends. Mechanized work verifies LTL-to-Büchi translations and model checkers [25]–[28]. Wang et al. verify an MLTL-to-regular-expression transformation, export executable code, and compare it with existing implementations [29]. For metric logics, Roohi and Viswanathan used PVS to correct an unsound release semantics in prior MITL decision procedures and prove supporting results for an adapted timed-automata construction under continuous-time signal se-

mantics [30]; Coq-based quantitative MTL monitoring addresses another task [31]. These targets differ from our pointwise infinite-word TBA theorem, but set precedents for proof-connected tools and for mechanization that changes a known result. Here the backend remains an explicit contract: no LTL translator is integrated or extracted, and Spot is not verified.

Event-B refinement patterns. Our work on Event-B patterns addresses their generation and refinement, including DSL-based pattern generation [32], patterns using weakest-precondition calculus [33], and pattern-based development using predicate transformers [34]. These works provide the Event-B pattern-generation context for our broader objective: given an MTL property, use its TBA translation as the input to a downstream Event-B pattern-generation step. The present paper establishes the MTL-to-TBA stage only. It gives no TBA-to-Event-B transformation or correctness result for generated Event-B patterns.

7 CONCLUSION

Under an explicit backend contract, we mechanize semantic preservation from well-formed core $MTL_{0,\infty}$ formulas to event-guarded timed Büchi automata with existential initial clocks. Clocked-LTL factorization and reset completion preserve the timed language with one clock per syntactically distinct primitive timed subformula.

Related constructions already have mathematical language-correctness results, including Li et al.’s full-fragment $MTL_{0,\infty}$ -to-TBA result. This paper contributes a Rocq-checked derivation for its real-bound, strictly increasing-time model and explicit backend boundary. The occurrence-indexed invariant and canonical reset-trace argument support that proof but are not shown to be reusable beyond this development. No new clock architecture, reset algorithm, or comparative efficiency result is established.

For the alarm, Rocq proves the backend contract for a manual Spot 2.16 transcription and instantiates the timed theorem to reject a fixed violating trace. An executable graph check improves auditability of that transcription, but its

parser is unverified; no integrated translator, timed checker, or performance study is provided. An immutable public artifact identifier must be added before final submission.

REFERENCES

- [1] A. Duret-Lutz, A. Lewkowicz, A. Fauchille, T. Michaud, E. Renault, and L. Xu, “Spot 2.0 — a framework for LTL and ω -automata manipulation,” in *Proceedings of the 14th International Symposium on Automated Technology for Verification and Analysis (ATVA’16)*, ser. Lecture Notes in Computer Science, vol. 9938. Springer, Oct. 2016, pp. 122–129.
- [2] P. Bulychyev, A. David, K. Larsen, and G. Li, “Efficient controller synthesis for a fragment of MTL($0, \infty$),” *Acta Informatica*, vol. 51, no. 3–4, pp. 165–192, Jan. 2014.
- [3] G. Li, P. G. Jensen, K. G. Larsen, A. Legay, and D. B. Poulsen, “Practical controller synthesis for MTL $_{0, \infty}$,” in *Proceedings of the 24th ACM SIGSOFT International SPIN Symposium on Model Checking of Software*, ser. SPIN 2017. New York, NY, USA: Association for Computing Machinery, 2017, pp. 102–111. [Online]. Available: <https://doi.org/10.1145/3092282.3092303>
- [4] E. Fares, J.-P. Bodeveix, H. Al-Aqrabi, and A. Salami, “Mechanically verified optimization and uppaal export for timed Büchi automata,” 2026, unpublished manuscript; no stable public version available.
- [5] D. D’Souza and P. Prabhakar, “On the expressiveness of MTL in the pointwise and continuous semantics,” *International Journal on Software Tools for Technology Transfer*, vol. 9, no. 1, pp. 1–4, 2007.
- [6] J. Ouaknine and J. Worrell, “On the decidability of metric temporal logic,” in *20th Annual IEEE Symposium on Logic in Computer Science (LICS 2005)*. IEEE Computer Society, 2005, pp. 188–197.
- [7] P. Bouyer, F. Chevalier, and N. Markey, “On the expressiveness of TPTL and MTL,” in *Foundations of Software Technology and Theoretical Computer Science (FSTTCS 2005)*, ser. Lecture Notes in Computer Science, vol. 3821. Springer, 2005, pp. 432–443.
- [8] G. Li, “CASAAL: A tool for translating MTL $_{0, \infty}$ into timed Büchi automata,” Online, 2017, available at the CASAAL tool page. [Online]. Available: <https://lcs.ios.ac.cn/~ligy/tools/CASAAL/>
- [9] R. Koymans, “Specifying real-time properties with metric temporal logic,” *Real Time Syst.*, vol. 2, no. 4, pp. 255–299, 1990. [Online]. Available: <https://doi.org/10.1007/BF01995674>
- [10] R. Alur, T. Feder, and T. A. Henzinger, “The benefits of relaxing punctuality,” *Journal of the ACM (JACM)*, vol. 43, no. 1, pp. 116–146, 1996.
- [11] D. Ničković and N. Piterman, “From MTL to deterministic timed automata,” in *Formal Modeling and Analysis of Timed Systems (FORMATS 2010)*, ser. Lecture Notes in Computer Science, vol. 6246. Springer, 2010, pp. 152–167.
- [12] T. Brihaye, M. Estièvenart, and G. Geeraerts, “On MITL and alternating timed automata over infinite words,” 2014. [Online]. Available: <https://arxiv.org/abs/1406.4395>
- [13] O. Maler, D. Nickovic, and A. Pnueli, “From MITL to timed automata,” in *Formal Modeling and Analysis of Timed Systems (FORMATS 2006)*, ser. Lecture Notes in Computer Science, vol. 4202. Springer, 2006, pp. 274–289.
- [14] T. Ferrère, O. Maler, D. Ničković, and A. Pnueli, “From real-time logic to timed automata,” *Journal of the ACM*, vol. 66, no. 3, 2019, art. no. 19.
- [15] T. Brihaye, G. Geeraerts, H.-M. Ho, and B. Monmege, “MightyL: A compositional translation from MITL to timed automata,” in *Computer Aided Verification*, R. Majumdar and V. Kunčák, Eds. Cham: Springer International Publishing, 2017, pp. 421–440.
- [16] “MITL2Timed: Modifying Oddoux and Gastin LTL2BA to timed automata generation,” GitHub repository. [Online]. Available: <https://github.com/yzh89/MITL2Timed>
- [17] M. M. Bersani, M. Rossi, and P. San Pietro, “A tool for deciding the satisfiability of continuous-time metric temporal logic,” *Acta Informatica*, vol. 53, no. 2, pp. 171–206, 2016.
- [18] R. Alur, L. Fix, and T. A. Henzinger, “Event-clock automata: A determinizable class of timed automata,” *Theoretical Computer Science*, vol. 211, no. 1–2, pp. 253–273, 1999.
- [19] J.-F. Raskin and P.-Y. Schobbens, “The logic of event clocks: Decidability, complexity and expressiveness,” *Journal of Automata, Languages and Combinatorics*, vol. 4, no. 3, pp. 247–286, 1999.
- [20] P. Bouyer, B. Srivathsan, and V. Vishwanath, “Model-checking real-time systems: Revisiting the alternating automaton route,” in *Foundations of Software Science and Computation Structures (FoSSaCS 2025)*, ser. Lecture Notes in Computer Science, vol. 15691. Springer Nature Switzerland, 2025, pp. 399–421.
- [21] S. Akshay, P. Contractor, P. Gastin, R. Govind, and B. Srivathsan, “TEMPORA: Efficient verification of metric temporal properties with past in pointwise semantics,” in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2026)*, ser. Lecture Notes in Computer Science, vol. 16505. Springer Nature Switzerland, 2026, pp. 640–659.
- [22] H.-M. Ho, S. N. Krishna, K. Madnani, R. Majumdar, and P. Pandya, “MightyPPL: Model checking MITL with past and Pnueli modalities,” in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2026)*, ser. Lecture Notes in Computer Science, vol. 16505. Springer Nature Switzerland, 2026, pp. 457–479.
- [23] —, “MightyPPL: Towards model checking MTL,” in *Quantitative Evaluation of Systems and Formal Modeling and Analysis of Timed Systems (QEST+FORMATS 2026)*, ser. Lecture Notes in Computer Science, vol. 16913. Springer, 2027, pp. 216–225.
- [24] —, “MightyPPL: Towards model checking MTL,” 2026. [Online]. Available: <https://arxiv.org/abs/2609.19073>
- [25] A. Schimpf, S. Merz, and J.-G. Smaus, “Construction of Büchi automata for LTL model checking verified in Isabelle/HOL,” in *Theorem Proving in Higher Order Logics (TPHOLs 2009)*, ser. Lecture Notes in Computer Science, vol. 5674. Springer, 2009, pp. 424–439.
- [26] J. Esparza, P. Lammich, R. Neumann, T. Nipkow, A. Schimpf, and J.-G. Smaus, “A fully verified executable LTL model checker,” in *Computer Aided Verification (CAV 2013)*, ser. Lecture Notes in Computer Science, vol. 8044. Springer, 2013, pp. 463–478.
- [27] A. Schimpf and P. Lammich, “Converting linear-time temporal logic to generalized Büchi automata,” *Archive of Formal Proofs*, 2014, https://isa-afp.org/entries/LTL_to_GBA.html, formal proof development.
- [28] B. Seidl and S. Sickert, “A compositional and unified translation of LTL into ω -automata,” *Archive of Formal Proofs*, https://isa-afp.org/entries/LTL_Master_Theorem.html, 2019.
- [29] Z. Wang, K. Kosaian, and K. Y. Rozier, “Formally verifying a transformation from MLTL formulas to regular expressions,” in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2025)*, ser. Lecture Notes in Computer Science, vol. 15696. Springer, 2025, pp. 254–275.
- [30] N. Roohi and M. Viswanathan, “Revisiting MITL to fix decision procedures,” in *Verification, Model Checking, and Abstract Interpretation (VMCAI 2018)*, ser. Lecture Notes in Computer Science, vol. 10747. Springer, 2018, pp. 474–494.
- [31] A. Chattopadhyay and K. Mamouras, “A verified online monitor for metric temporal logic with quantitative semantics,” in *Runtime Verification (RV 2020)*, ser. Lecture Notes in Computer Science, vol. 12399. Springer, 2020, pp. 383–403.
- [32] E. Fares, J.-P. Bodeveix, and M. Filali, “Pattern-based refinement generation through domain specific languages,” in *Rigorous State-Based Methods (ABZ 2023)*, ser. Lecture Notes in Computer Science, vol. 14010. Springer, 2023, pp. 35–42.
- [33] —, “Correct pattern-based development through refinements and weakest preconditions calculus,” in *Formal Aspects of Component Software (FACS 2024)*, ser. Lecture Notes in Computer Science, vol. 15189. Springer, 2024, pp. 59–78.
- [34] —, “Correct pattern-based development through refinements and predicate transformers,” *Science of Computer Programming*, vol. 255, p. 103550, 2027.